神经网络是怎么"学会"的

PyTorch 理论精读 · 00-2 章配套讲稿
整理于 2026-10-09

一、神经元：加权打分（对应你的 nn.Linear）

教材的猫例子：得分 = w1 · x1 + w2 · x2 + w3 · x3 + b。你早就写过它的真身：

```
model = nn.Linear(2, 2)   # 就是 2 个神经元，每个做 w1 · x1 + w2 · x2 + b
```

这里往深走一步：**为什么"打分"要用乘法加法，而不是别的？** 因为线性运算是唯一能被高效求导、又能层层组合的运算。几百万维的山要下山，每一步都要求梯度——而 w · x 对 w 的导数就是 x，干净、便宜、可组合。整个深度学习大厦建立在这个"可微"的地基上。

还有个直觉：**权重就是记忆**。训练前 w 是随机数（模型什么都不懂），训练后 w 里存着从数据里学到的规律。"大模型有上万亿参数"的意思是：它把从整个互联网学到的统计规律，压缩存进了上万亿个数里。

二、激活函数：没有它，一百层等于一层（第 03 章的面试题）

教材说"直线套直线还是直线"，给你把这句话变成数学：

层1: y = W1 · x + b1
层2: z = W2 · y + b2
代入: z = W2 · (W1 · x + b1) + b2 = (W2 · W1) · x + (W2 · b1 + b2)
↑ 一个新的 W ↑ 一个新的 b

两层线性 =
一层线性。所以**深度必须配非线性**，否则堆层数毫无意义。

ReLU 为什么能成为主流？它简单到粗暴（负数掐
0），但正因为简单：求导不是 0 就是
1，**梯度不会在层层相乘中消失**。早期的 sigmoid 在两端导数趋近
0，十几层网络梯度乘起来就小得没法训练了——这叫"梯度消失"，
ReLU 是破局的关键之一（也是教材第 6
节说的"算法条件到位"之一）。

你的 practice 里 nn.ReLU() 夹在 fc1 和 fc2 之间，现在你知道它不只是"调料"，是让两层网络真正比一层强的结构件。

三、损失函数：把"错"变成一个可求导的数

教材的例子很直观，往深走一步——**为什么分类用交叉熵而不是"错误率"?**

错误率（分对/分错的比例）看起来最直接，但它是个**阶梯函数**：
预测从 0.51 改到 0.49，错误率跳变，中间梯度是
0。蒙眼下山的人脚踩在平地上，完全不知道往哪走。

交叉熵的巧妙在于**连续且对信心敏感**：你在任务 2
亲手算过——预测对且自信（p=0.66）时 loss=0.42；换标签后
loss=2.32。而且 -log(p) 有个漂亮性质：预测越自信地错（p→0）
，损失趋向无穷大——重罚"自信的错误"，梯度巨大，模型会被猛地拉回来。这就是训练前期 loss 下降快的原因。

四、梯度下降：你的 lr 实验就是这一节的实证

下山类比教材讲得不错，我补一个精确版，把你任务 4 的现象彻底解释掉：

对二次损失 $L = (3w - 9)^2$，梯度是 $18(w-3)$，参数更新是：

$$w_{new} = w - lr \cdot 18(w - 3)$$
$$= 3 + (1 - 18 \cdot lr)(w - 3) \quad \leftarrow \text{每一步，离目标的距离乘以 } (1-18lr)$$

生死线就在这：**$|1 - 18 \cdot lr| < 1$ 才收敛，即 $lr < 0.111$**。

- lr=0.05：系数 0.1，每步距离缩到 1/10 → 3 步收敛（你看到的 36→0.36→0）
- lr=0.1：系数 -0.8，左右横跳但幅度缩小 → 慢速收敛
- lr=0.2：系数 -2.6，每步跳到对面且距离放大 2.6 倍 → 36→243→1645，指数爆炸

分类任务里 lr=10 没炸，是因为交叉熵在可分数据上的"山坡"形状不同——越接近谷底坡度越缓，天然有缓冲。**所以学习率没有万能值，它取决于损失曲面的形状**——这就是为什么调度器（先大后小）有效：远处大步赶路，近处小步细调。

五、反向传播：链式法则 + 动态规划

教材的"追责"类比很准。往深走一步：反向传播本质上是**链式法则的聪明记账**。

朴素的链式法则有个问题：从 loss 到每个参数的导数路径有大量重复计算。反向传播的 insight 是——**从后往前算，把每层的中间导数存下来复用**，每条路径只算一次。这在算法上就是动态规划。

代价是什么？**必须把所有中间结果（前向的每层输出）存着**，等反向时用——这就是为什么：

- 训练比推理费显存（推理不用存中间结果）
- 验证时必须 torch.no_grad()（不建图 = 不记账 = 省下全部中间存储）
- loss.backward() 后图就释放了（账记完就撕），再 backward 一次会报"图已释放"

你在 practice02 里见过的这些"规矩"，全部是反向传播这条记账流水线的直接推论。

六、mini-batch：噪声不是缺陷，是特性

教材讲了"抽一小批估计方向"，补两个 deeper 的点：

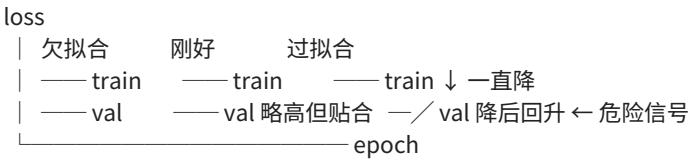
1. batch 大小的三极权衡

batch	梯度质量	速度	泛化
全量	最准	慢到不可用	反而容易过拟合
32~256（主流）	有噪声但方向对	GPU 并行吃满	噪声带来正则化效果
1（纯随机）	噪声巨大	并行度浪费	太跳，难收敛

2. 为什么噪声有助于泛化？ 全量梯度会精确地走向当前训练集的最低点——包括那些只属于训练集的"假谷底"（背答案）。mini-batch 的噪声让模型在这些尖锐的小谷底里站不住脚，只有**又宽又平的谷底**（对数据扰动不敏感的真规律）才能留住它。这就是"随机梯度下降的隐式正则化"——学术上至今仍在研究的课题。

七、过拟合：这章最重要的一节

背答案 vs 真学会的类比之外，给你一个可操作的判断框架——**看训练/验证两条曲线的相对位置**：



- **欠拟合**：两条都高 → 加大模型/多训/降正则
- **过拟合**：train 降 val 升 → 加数据/数据增强（你下一章要学的！）/Dropout/weight decay/早停

注意第 04 章说的"只存 best"就是**早停**的朴素版：验证集最准的那个 checkpoint，就是过拟合开始前的最后一刻。

还有三组概念的精确区分，面试常问：

- **验证集 vs 测试集**：验证集用来"做决策"（调超参、选模型），测试集只在最后用一次。用测试集调参 = 偷看高考答案，成绩作废
- **Dropout 训练开/评估关**：训练时随机掐神经元是在"破坏记忆路径"防背诵；评估时要全体神经元稳定输出——这就是 model.eval() 存在的意义（你在任务 5 亲手验证过）
- **weight decay（L2 正则）**：给损失加一项"参数大小的惩罚"，逼模型用小权重——大权重往往对应着对训练集细节的过度敏感。AdamW 的 W 就是把这项改进做对

八、小结成一张图

一个神经元：w·x+b（打分）　　← 你的 nn.Linear
　↓ 叠层+ReLU（拐弯）　　← 你的 TinyNet
前向：算出预测
　↓
损失函数：错多少（连续可导的尺子）　← 你的 CrossEntropyLoss
　↓
反向传播：每个参数分到自己的责任　← 你的 loss.backward()
　↓
梯度下降：w -= lr·grad（下山一步）　← 你的 optimizer.step()
　↑ 用 mini-batch 循环，每轮 zero_grad ← 你的训练循环
监控：train loss ↓ 且 val acc ↑ = 真学会 ← 你的 eval+no_grad 验证

这一章如果你能给别人讲明白"为什么 lr 太大会炸"（用 1-18·lr 那个式子）和"为什么验证集是照妖镜"，理论就真扎实了。